Taller del Segundo Corte: Docker
Informe de práctica (Parte 2)

Arquitectura de Software

Presentado por:

Christian Javier Benitez Castro - T00082581

Docente:

Luis Giovanny Carreño Ortiz

Universidad Tecnológica de Bolívar
Cartagena de Indias

16/10/2026

Introducción

En esta práctica del segundo corte de Arquitectura de Software utilicé Docker para poner en funcionamiento el proyecto **biblioteca-api**, una API REST desarrollada en Go y conectada a MySQL. Seguí la guía del archivo README.md entregado por el docente desde la terminal CMD de Windows: preparé el repositorio, configuré las variables del entorno, construí los contenedores y comprobé las operaciones de autenticación y consulta de libros. Además, identifiqué las dificultades encontradas y su relación con los conceptos estudiados en la primera parte del taller.

Objetivo de la práctica

Ejecutar el proyecto de biblioteca por medio de Docker Compose, comprobar que la API y su base de datos se comunican correctamente y documentar los comandos, resultados y evidencias obtenidas.

1. Fase de preparación y requisitos previos

Trabajé en Windows mediante CMD, con Docker Desktop y su motor de contenedores Linux. La verificación mostró **Docker 29.8.0** y **Docker Compose v5.5.1**. Revisé también la existencia de contenedores anteriores y los detuve mediante **docker compose stop** para evitar interferencias con el ejercicio nuevo. La salida quedó registrada en la evidencia 1.

Para obtener el código utilicé Git con el comando **git clone https://github.com/carrenoutb/biblioteca-api.git biblioteca-api-taller**, entré a la carpeta y revisé su contenido con **dir /a**. Se observaron los archivos Dockerfile, docker-compose.yml, .env.example y README.md, además de las carpetas del código Go y las migraciones. El clonado está documentado en la evidencia 2.

Respecto a las herramientas del equipo, Docker Desktop permite administrar los contenedores, Git sirve para traer el repositorio y WSL 2 es el entorno Linux que puede utilizar Docker Desktop en Windows. No es necesario instalar manualmente MySQL ni Go para ejecutar esta guía: las imágenes de Docker aportan el entorno de ejecución. En las capturas no se documenta una instalación nueva de Git o de WSL 2, sino el uso posterior de las herramientas.

Archivos importantes del proyecto

Archivo	Función durante el ejercicio
Dockerfile	Indica cómo compilar el programa Go y preparar la imagen de la API.
docker-compose.yml	Describe los servicios mysql y api, redes, puertos, volúmenes y orden de arranque.
.env.example / .env	Plantilla y archivo local con los parámetros de conexión y la configuración de la aplicación.
migrations/	Scripts SQL que preparan la estructura y los datos iniciales de una base nueva.
docs/	Archivos necesarios para mostrar la documentación Swagger.

2. Fase de configuración y puesta en marcha

En la carpeta del proyecto ejecuté **copy .env.example .env** y confirmé la presencia de ambos archivos mediante **dir /a .env***. El primer intento de copia encontró un archivo .env anterior y acepté reemplazarlo. Después validé la configuración con **docker compose config --quiet**. La validación solo indicó que el atributo

version de Compose está obsoleto, lo cual no impedía continuar.

Variables del archivo .env

Variable	Propósito
DB_HOST	Nombre del servicio de la base de datos dentro de la red Docker; en este proyecto es mysql.
DB_PORT	Puerto interno de MySQL, 3306, usado desde el contenedor de la API.
DB_USER	Usuario con el que la aplicación se autentica ante MySQL.
DB_PASSWORD	Contraseña correspondiente al usuario de la base de datos.
DB_NAME	Base de datos con la que trabaja la API.
JWT_SECRET	Secreto utilizado para firmar los tokens de autenticación.
JWT_EXPIRATION_HOURS	Duración del token expresada en horas.
APP_PORT	Puerto donde la API escucha peticiones HTTP dentro de su contenedor.

Luego ejecuté **docker compose up --build**. Docker construyó la imagen de la API y creó los contenedores, pero MySQL no pudo ocupar el puerto 3306 de Windows porque ya estaba en uso. Para solucionarlo cambié el mapeo externo en docker-compose.yml de **3306:3306** a **3307:3306**, sin alterar el puerto interno que utiliza la API. Al ejecutar de nuevo el comando, MySQL informó que estaba listo para aceptar conexiones y la API mostró “Conexión a MySQL establecida correctamente” y “Servidor escuchando en el puerto 8080”. Esto se observa en las evidencias 3 y 4.

3. Fase de verificación

Abrí otra ventana de CMD y ejecuté **docker compose ps**. El resultado mostró el contenedor biblioteca_api en estado Up, expuesto en el puerto 8080, y biblioteca_mysql en estado Up (healthy), con MySQL publicado en el puerto externo 3307. La evidencia 5 registra estos estados.

Después accedí a **http://localhost:8080/swagger/index.html**, donde se visualizó la documentación de Biblioteca API, incluidas las rutas de registro, inicio de sesión, libros y préstamos (evidencia 6). Realicé el flujo de prueba indicado por el README: **POST /api/auth/register**, **POST /api/auth/login** y **GET /api/books** utilizando autenticación JWT.

En el registro obtuve el código **201** y al iniciar sesión obtuve el código **200**. La evidencia 7 presenta el formulario de registro completo y una captura adicional de la sección de Swagger que documenta **201 Created** (es un ejemplo de respuesta, no la respuesta real del servidor). La evidencia 8 presenta el formulario de login y una captura real de **Server response: 200**, con los datos del usuario y su token JWT. Por último, la evidencia 9 muestra **GET /api/books** con código **200** y libros como *Cien años de soledad*, *Clean Code* y *El principito*. Se conservaron todos los datos visibles de las capturas originales.

4. Problemas identificados y soluciones

Conflicto del puerto de MySQL (ocurrió). El primer arranque falló con el mensaje de que no se podía asignar el puerto TCP 3306. El servicio MySQL necesitaba publicarlo, pero ese puerto ya estaba ocupado en Windows. Modifiqué el mapeo a 3307:3306 y repetí el arranque. Se conservó 3306 como puerto interno, por

lo que el archivo .env no requirió cambiar DB_PORT.

Comprobación de Git (preparación). Al inicio existía la duda de si CMD reconocía Git, porque durante un intento anterior el comando no había funcionado. La forma de diagnosticarlo es ejecutar git --version; si no se reconoce, instalar Git for Windows y abrir una terminal nueva. En esta práctica pude ejecutar git clone satisfactoriamente, por lo que no se presenta como un fallo confirmado en la captura.

WSL 2 y Docker Desktop (requisito que puede causar fallos). En Windows, el motor Linux de Docker Desktop puede necesitar WSL 2 actualizado y habilitado. Si el motor no inicia, se puede revisar wsl --version y wsl --status, completar la configuración y reiniciar el equipo. No hubo evidencia de una falla de WSL durante la ejecución del taller.

Advertencia de version (ocurrió, sin bloquear). Compose mostró que la clave version de docker-compose.yml está obsoleta y se ignora. Se puede retirar esa línea del archivo para evitar la advertencia, sin cambiar el comportamiento de la aplicación.

Ausencia del archivo .env (problema posible). Si el archivo falta y Compose contiene env_file: .env, el proyecto puede detenerse antes de iniciarse. Se corrige copiando la plantilla .env.example a .env y comprobando su existencia. En esta ejecución el archivo sí se preparó.

5. Preguntas de comprensión

1. ¿Qué diferencia hay entre una imagen y un contenedor?
La imagen es una plantilla que almacena los elementos necesarios para ejecutar una aplicación; el contenedor es una instancia creada a partir de ella. En este proyecto, mysql:8.0 es la imagen descargada y biblioteca_mysql es el contenedor que se inicia para almacenar los datos. Del mismo modo, el Dockerfile permite construir la imagen propia de biblioteca-api, que luego corre en el contenedor biblioteca_api.

2. ¿Qué hace docker compose que no harías con docker run por separado?
Compose describe los servicios de una aplicación en un mismo archivo y administra en conjunto sus contenedores, red, puertos, volúmenes, variables y dependencias. Con docker run podría iniciar MySQL y la API mediante órdenes separadas, pero tendría que coordinar cada configuración de forma manual. Aquí Compose permite levantar ambos mediante un solo comando y esperar el estado saludable de MySQL.

3. ¿Por qué DB_HOST=mysql y no 127.0.0.1?
Docker Compose crea una red en la que cada servicio puede localizar a otro por su nombre. Desde el contenedor de la API, mysql identifica al servicio de la base de datos. En cambio, 127.0.0.1 apunta al propio contenedor de la API, donde no se está ejecutando MySQL, y por eso no serviría para esta conexión.

4. ¿Qué borra docker compose down -v y cuándo lo usarías?
Detiene y elimina los contenedores y la red del proyecto, y con la opción -v elimina también sus volúmenes administrados por Compose, incluidos los datos persistentes de MySQL. Lo utilizaría solamente cuando necesite reiniciar la práctica desde una base de datos vacía y sepa que puedo perder la información anterior. Sin -v, normalmente el volumen se conserva.

5. ¿Qué pasa si cambio DB_PASSWORD en .env y no en docker-compose.yml? ¿Y al revés?
DB_PASSWORD es la contraseña que la API utiliza para conectarse; en docker-compose.yml, MYSQL_PASSWORD define la contraseña que se asigna al usuario de MySQL durante la inicialización de la base. Si únicamente cambia una de ellas, las credenciales pueden dejar de coincidir y la API fallará al autenticarse. Además, cambiar MYSQL_PASSWORD en Compose no modifica automáticamente la contraseña de una base MySQL ya inicializada en un volumen persistente: en ese caso habría que actualizar la

contraseña dentro de MySQL y ajustar la configuración de la API.

Conclusión

Esta práctica me permitió comprobar el funcionamiento de Docker en un proyecto real, no solamente con comandos de ejemplo. Aprendí que el Dockerfile sirve para crear la imagen y que Compose facilita ejecutar una API junto con una base de datos y comprobar su comunicación. También entendí la importancia de los puertos: el conflicto del 3306 se resolvió cambiando el puerto de Windows, sin modificar el utilizado entre contenedores. Considero que esta forma de trabajar es útil en proyectos universitarios y en equipos de desarrollo porque facilita repetir el mismo entorno, identificar problemas y verificar el resultado mediante pruebas y evidencias.

Fuentes de consulta

Carreño, L. G. (2026). *Taller del 2do corte — Docker*. Guía de la asignatura Arquitectura de Software.

Repositorio del curso. (2026). *biblioteca-api: README.md*. https://github.com/carrenoutb/biblioteca-api

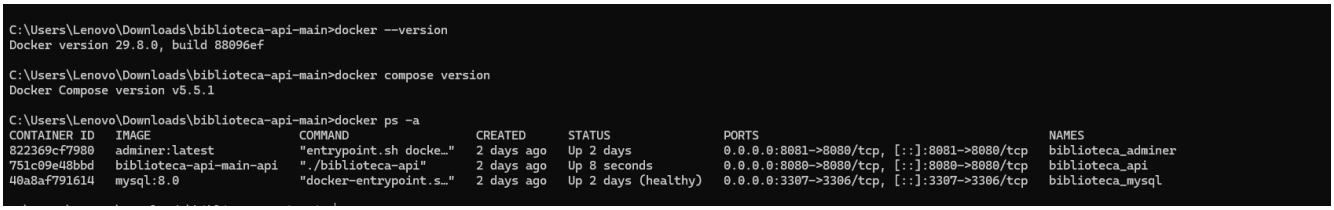
Docker Inc. (s. f.). *Docker Compose*. https://docs.docker.com/compose/

Anexo. Evidencias de ejecución

Se adjuntan las nueve capturas originales y dos imágenes complementarias, todas a color y sin ocultar los datos visibles. La captura complementaria de 201 corresponde al ejemplo de respuesta documentado por Swagger, mientras que la de 200 muestra la respuesta real del login. Para acreditar de manera directa el registro ejecutado haría falta una captura del apartado Server response del registro.

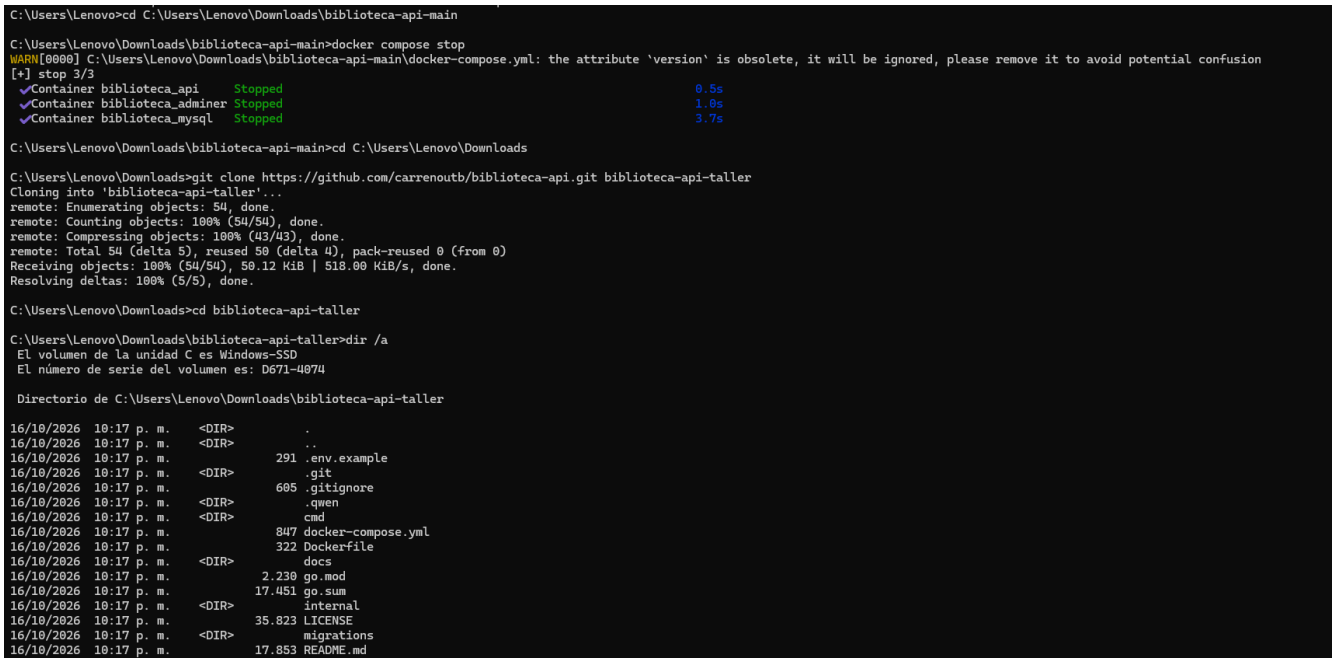
Evidencia 1. Verificación de Docker

La terminal muestra Docker 29.8.0, Docker Compose v5.5.1 y los contenedores previos.



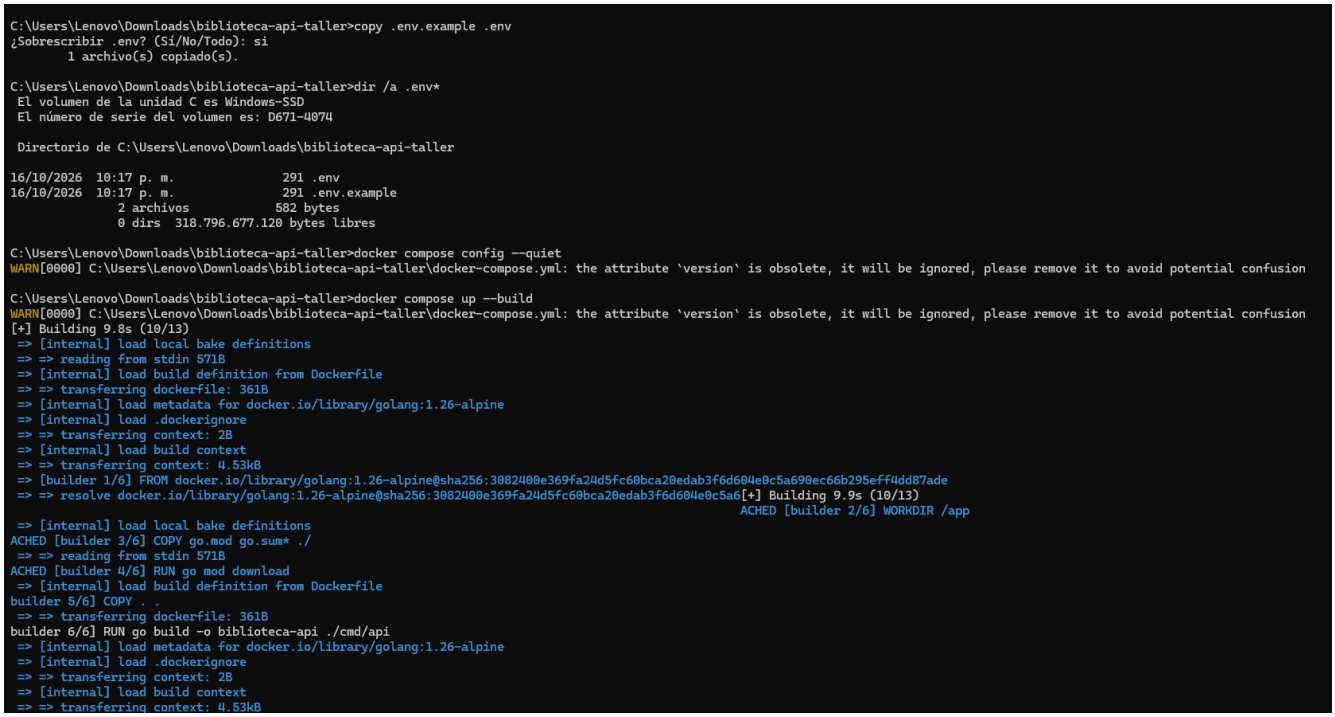
Evidencia 2. Repositorio clonado

Se detuvieron los contenedores anteriores, se clonó el repositorio del curso y se revisaron sus archivos.



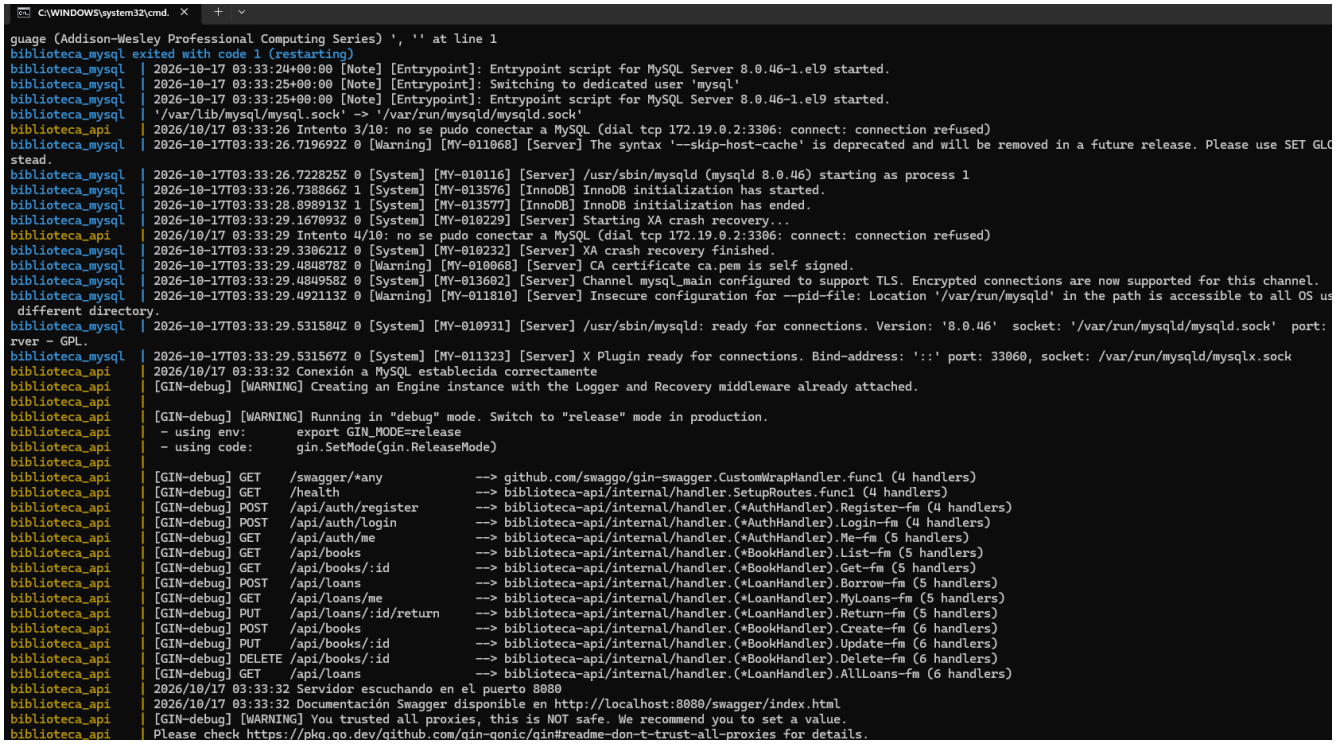
Evidencia 3. Entorno y construcción de imagen

Se copió .env.example a .env, se validó Compose y se inició la compilación; aparece el primer conflicto del puerto 3306.



Evidencia 4. Inicio de los contenedores

Después del ajuste de puerto, MySQL quedó listo y la API informó que escuchaba en 8080.



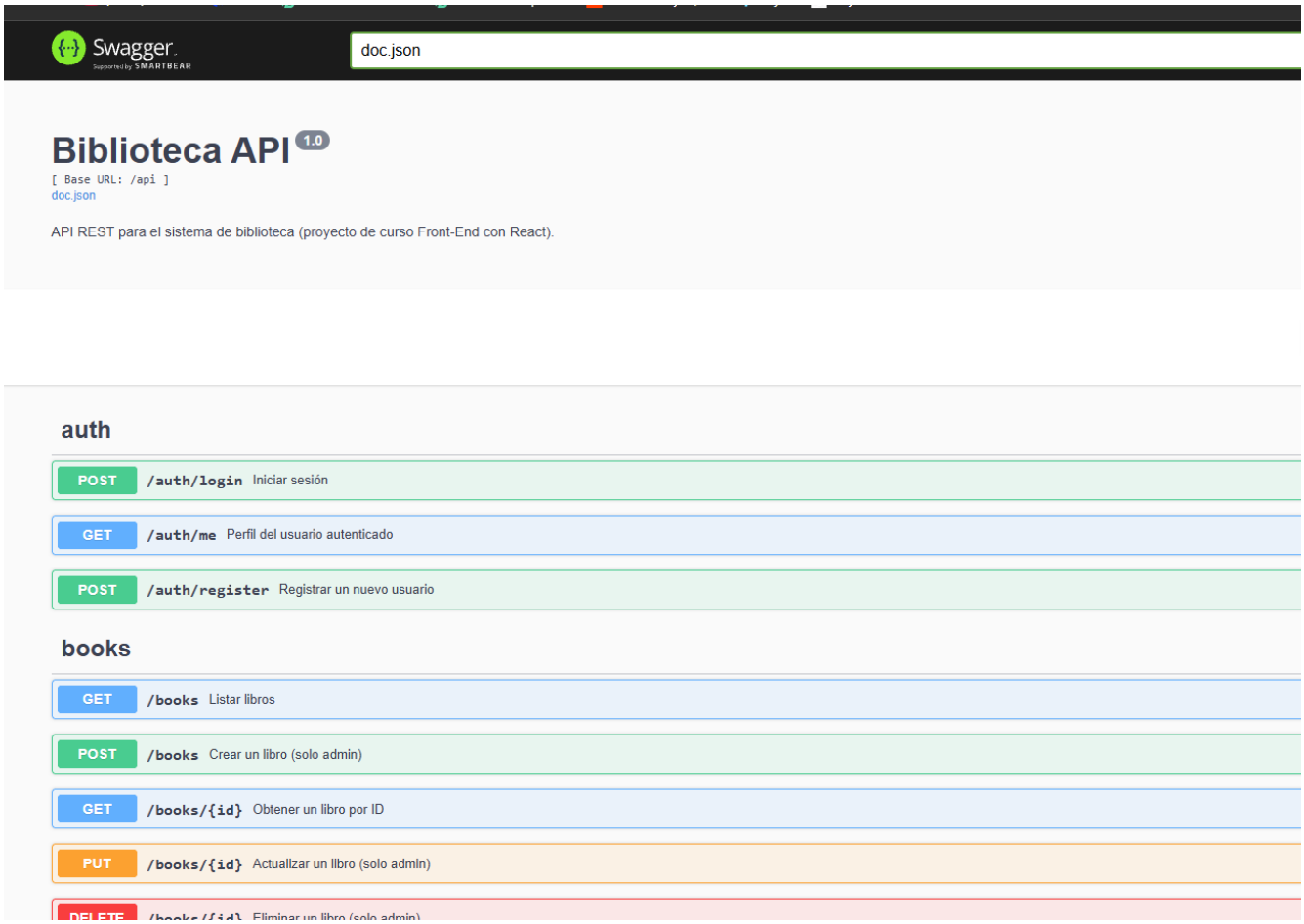
Evidencia 5. Estado con docker compose ps

Docker Compose muestra el servicio API activo y MySQL con estado healthy.



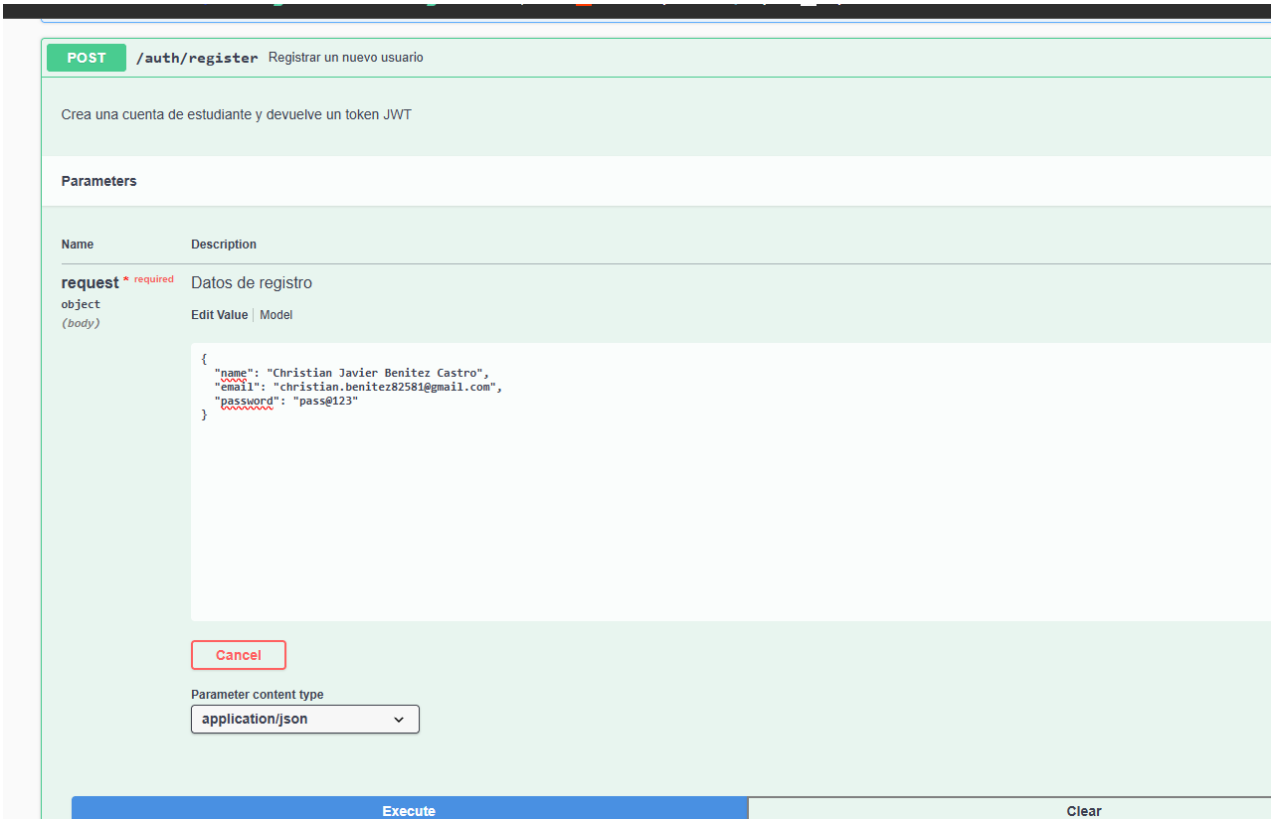
Evidencia 6. Documentación Swagger

Documentación Swagger disponible en el navegador con las rutas de autenticación y libros.



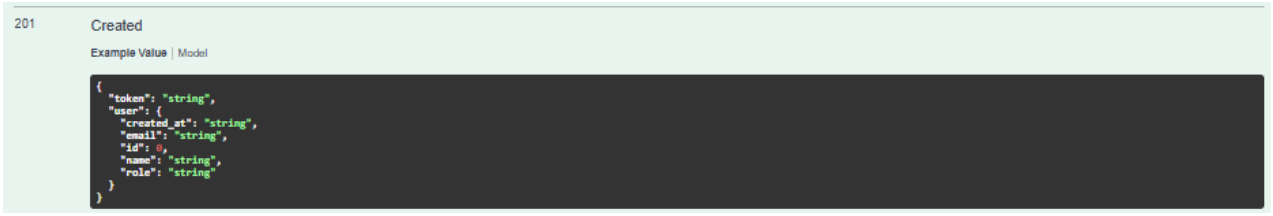
Evidencia 7. Registro de usuario

Formulario de registro con nombre, correo y contraseña de prueba visibles, tal como en la captura original.



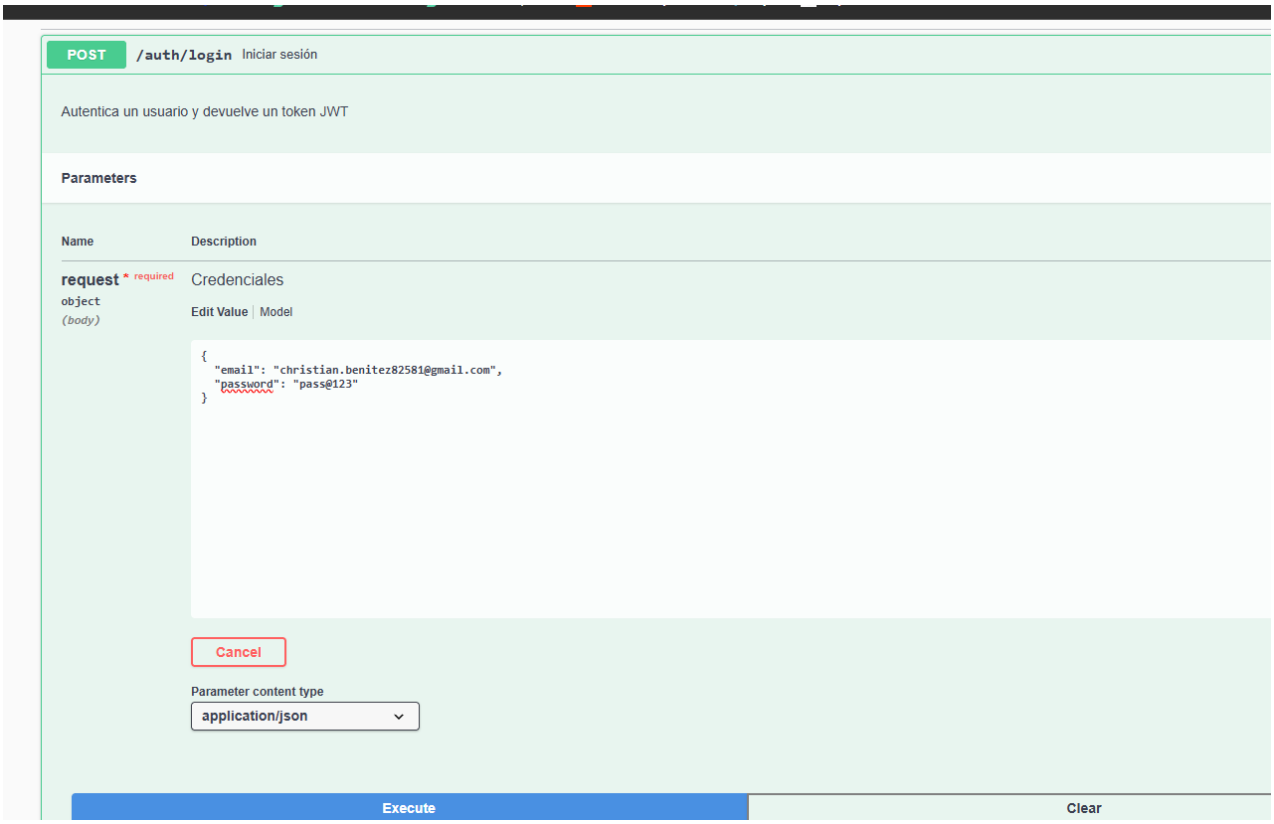
Captura complementaria: Código 201 Created en la documentación de Swagger

Esta vista corresponde a la respuesta de ejemplo documentada por Swagger; no es una captura de la respuesta real de la solicitud.



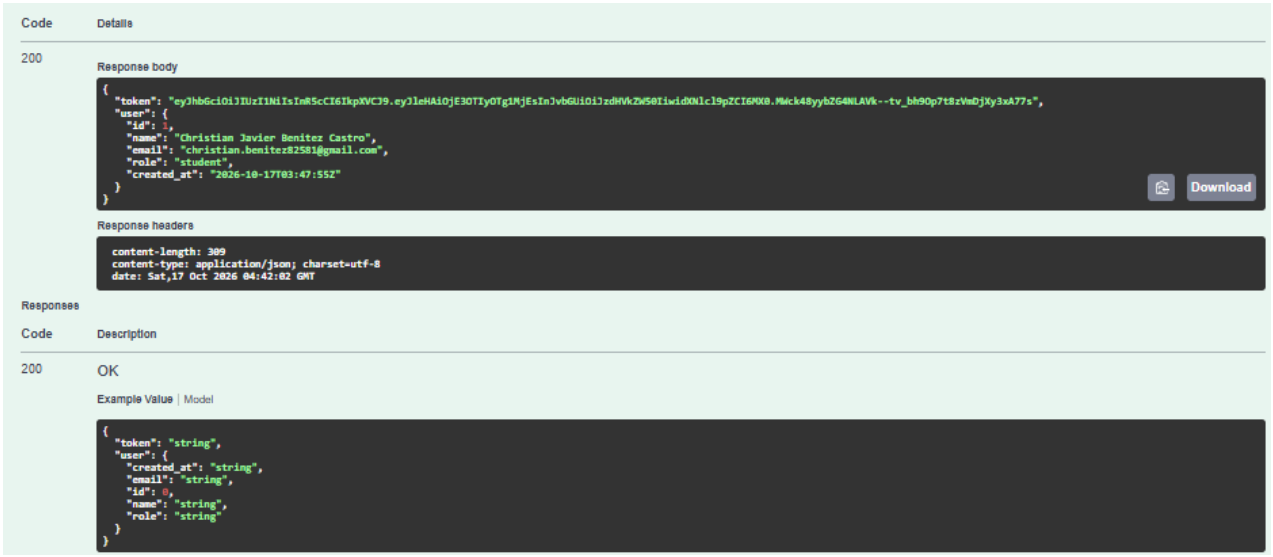
Evidencia 8. Inicio de sesión

Formulario de login con correo y contraseña de prueba visibles, tal como en la captura original.



Captura complementaria: Respuesta real HTTP 200 del login

La sección Server response muestra el código 200, el usuario y el token JWT devuelto.



Evidencia 9. Consulta de libros con HTTP 200

Consulta GET /api/books con HTTP 200, datos de varios libros y la petición original completa.

